

```

Administrator: Command Prompt - simplecv

SimpleCV 1.3.0 [interactive shell - http://simplecv.org]

Commands:
  "exit()" or press "Ctrl+ B" to exit the shell
  "clear" to clear the shell screen
  "tutorial" to begin the SimpleCV interactive tutorial
  "example" gives a list of examples you can run
  "forums" will launch a web browser for the help forums
  "walkthrough" will launch a web browser with a walkthrough

Usage:
  dot complete works to show library
  for example: Image().save("~/tmp/test.jpg") will dot complete
  just by touching TAB after typing Image().

Documentation:
  help(Image?), ?Image, Image?, or Image<?> all do the same
  "docs" will launch webbrowser showing documentation

SimpleCV:1>

```

Figure 2: SimpleCV console

```
SimpleCV:1> help(Image)
```

The successful response for the above command would be:

```

Help on class Image in module SimpleCV.ImageClass:
class Image
|  **SUMMARY**
|  The Image class is the heart of SimpleCV and allows you to
|  convert to and
|  from a number of source types with ease. It also has
|  intelligent buffer
|  management, so that modified copies of the Image required
|  for algorithms
|  such as edge detection, etc can be cached and reused when
|  appropriate.
|
|  Images are converted into 8-bit, 3-channel images in RGB
|  colorspace. It will
|  automatically handle conversion from other representations
|  into this
|  standard format. If dimensions are passed, an empty image
|  is created.
|
|  **EXAMPLE**
|
|  >>> i = Image("/path/to/image.png")
|  >>> i = Camera().getImage()
|
|  You can also just load the SimpleCV logo using:
|
|  >>> img = Image("simplecv")
|  >>> img = Image("logo")
|
|  Or you can load an image from a URL:
|
|  much more here ...

```

A detailed tutorial on interactive shells is provided at <http://tutorial.simplecv.org/en/latest/examples/shell.html>

SimpleCV HelloWorld

As stated in the beginning of this article, the core idea of SimpleCV is to make machine vision simple. A HelloWorld example is shown below:

```

from SimpleCV import Camera
# Initialize the camera
cam = Camera()

# Loop to continuously get images
while True:
    # Get Image from camera
    img = cam.getImage()

    # Make image black and white
    img = img.binarize()
    # Draw the text "Hello World" on image
    img.drawText("Hello World OSFY !!!")
    # Show the image
    img.show()

```

As can be observed from the code sample, SimpleCV provides direct functions for performing the vision tasks. The first step is to get the image source from a camera, which can be done using the `cam.getImage()`. To convert into black and white, the direct function `img.binarize()` should be used. Similarly, drawing the text also can be carried out using the simple method `img.drawText()`. The output image can be displayed using `img.show()`.

SimpleCV provides mechanisms to capture the live camera feed using simple methods as shown below:

```

from SimpleCV import Camera
cam = Camera()
cam.live()

```

An example of machine learning

Apart from performing the simple image manipulation tasks as shown in the HelloWorld example, it allows us to perform comparatively advanced tasks such as image classification. The classification is based on the machine learning. A sample code for performing classification is available at the official code repository of SimpleCV (https://github.com/sightmachine/SimpleCV/blob/develop/SimpleCV/examples/machine-learning/machine-learning_nuts-vs-bolts.py). This example classifies the given image into 'Nuts' and 'Bolts' with Scikits-learn library. The image features such as "area", "height" and "width" are extracted from the images for the classification purpose:

SimpleCV Blob handling

Blobs are regions of similar pixels. Blobs are used to detect items of interest in the image. Once the blobs are identified, many operations shall be performed as shown in Figure 3.